PROIECT 04 / GHID DE DEZVOLTARE

ProcureAI

Sistem de achiziții cu mai mulți agenți

React + Vite • .NET 10 • PostgreSQL / EF Core
Docker / Docker Compose • MinIO / S3 • Microsoft Foundry

Categorie: Text / documente / imagini; audio și video opțional

Scop: Descrii necesarul, încarci oferte și rulezi agenți cu roluri distincte. Aplicația normalizează ofertele, aplică o grilă deterministă și produce o recomandare cu surse.

Arhitectură	Cerință
Backend simplu	Monolit modular; un API și o singură bază de date.
5 containere permanente	web, api, ai, db, minio; migrate este one-shot.
Fără utilizatori	Fără login/roluri, billing, dashboard sau administrare de business.
Foundry obligatoriu	Nucleu AI real + adaptoare de laborator pentru familiile AI-103.
RAG opțional/configurabil	Azure AI Search, pgvector, File Search și agentic retrieval.

Versiunea 1.0 • 11 octombrie 2026

Specificație de implementare și fragmente de referință, nu repository complet compilat. Include API, EF, integrare AI, Dockerfile-uri, Compose, evaluare și criterii de acceptare. Codul necesită completarea entităților/serviciilor și testare .NET/Docker locală.

Cuprins

01 / Produsul și cerințele exacte 3
02 / Arhitectura: monolit modular + gateway AI 4
03 / Structura soluției și modulele backend 5
04 / Stack, versiuni și biblioteci 6
05 / Schema PostgreSQL: nucleu comun și domeniu 7
06 / Entități și configurare EF Core 8
07 / DbContext, tranzacții și migrații 10
08 / Contractele API publice și interne 11
09 / Upload și stocarea fișierelor în S3 12
10 / Joburi durabile fără Redis sau RabbitMQ 13
11 / Extragere și normalizare pentru acest proiect 14
12 / Strategii RAG interschimbabile 15
13 / Indexul Azure AI Search 16
14 / Retrieval, ranking și validarea surselor 17
15 / Prompturi, JSON și validarea rezultatului 18
16 / AI Gateway .NET: contract REST și autentificare 19
17 / Embeddings și varianta pgvector 20
18 / Content Understanding: fișiere binare și operații async 21
19 / Agenți Foundry și tools specifice domeniului 22
20 / Toolbox, MCP, OpenAPI și tool-uri găzduite 23
21 / Modele, memorie, multi-agent și optimizare 24
22 / Imagini, video, sunet și text: adaptoare Foundry 25
23 / Frontend React + Vite: un singur ecran 26
24 / Dockerfile-uri pentru cele patru aplicații 28
25 / Docker Compose: cinci containere + migrare one-shot 30
26 / Configurare, secrete și feature flags 32
27 / Bootstrap și pașii de dezvoltare 33
28 / Securitate, observabilitate și controlul costurilor 34
29 / Evaluări, teste și CI/CD pentru domeniu 35
30 / Diagnostic: probleme probabile și remediere 36
31 / Ordinea implementării și acoperirea AI-103 37
32 / Fluxul end-to-end și limita fiecărui pas 38
33 / Logica .NET specifică și reguli deterministe 40
34 / Scenarii de test și acceptare end-to-end 42
35 / Frontend și pașii concreți de dezvoltare 43
36 / Surse oficiale și limitele livrabilului 44

01 / Produsul și cerințele exacte

Descrii necesarul, încarci oferte și rulezi agenți cu roluri distincte. Aplicația normalizează ofertele, aplică o grilă deterministă și produce o recomandare cu surse.

Funcționalitatea strict necesară

• Un singur ecran: brief, oferte, criterii și Run.

• Agenți separați: necesar, furnizori/oferte, comparație și recomandare.

• Toți agenții rulează în același container AI; backend-ul deține starea.

• Selecția utilizează documentele/fixture-urile încărcate și opțional căutare web citată.

• Nu include ERP, stocuri, comenzi de cumpărare, email sau portal furnizori.

Cum se acoperă Microsoft Foundry

Nucleul folosește model Foundry, tools și procesare/extragere relevantă temei, cu safety, evaluare și tracing. Familiile AI-103 sunt mapate în capitolul 31; adaptoarele Speech/Language/Translator și generare media sunt exerciții activabile în același container AI.

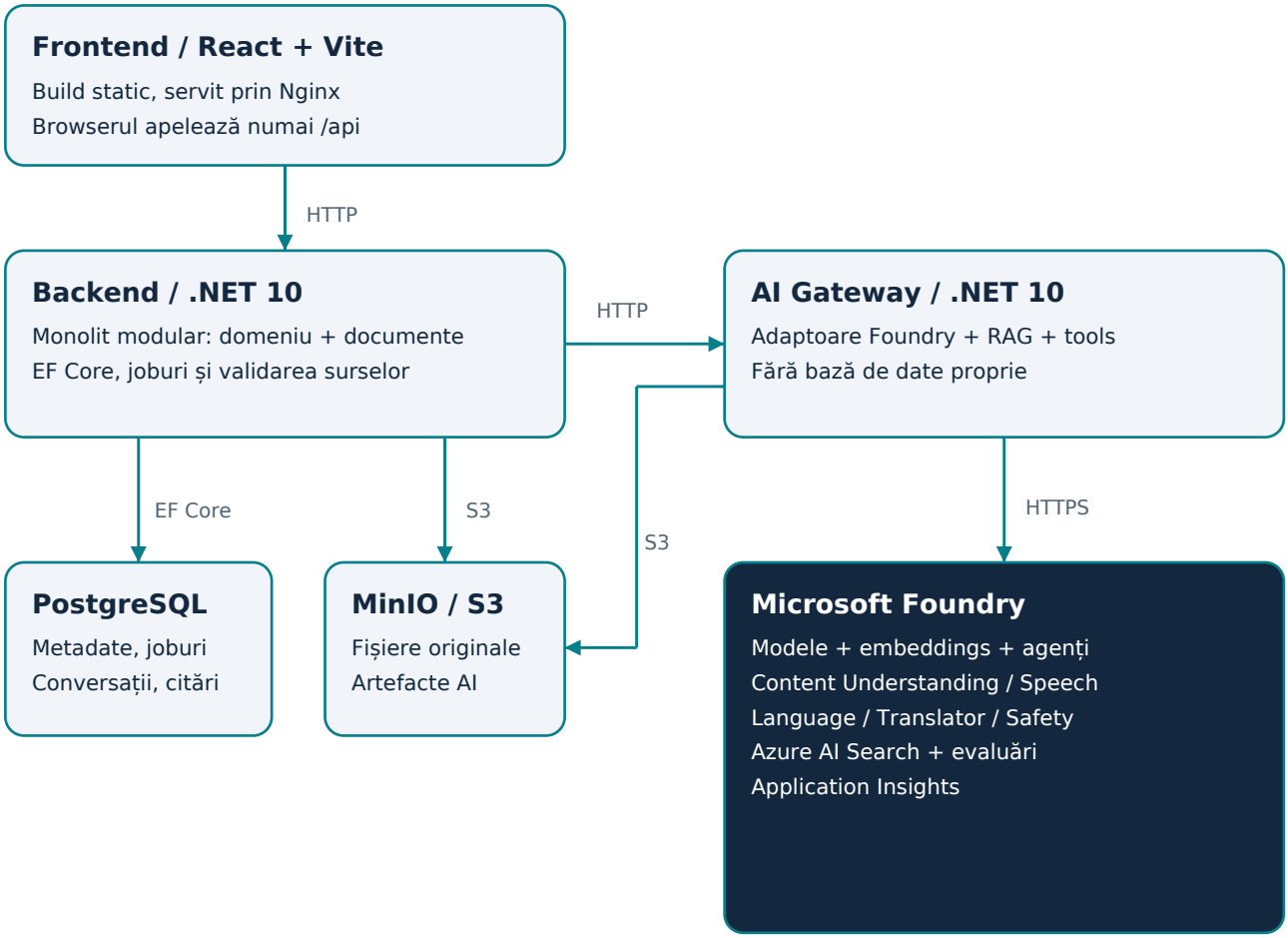
„Tot ce este în Foundry” înseamnă aici acoperirea familiilor de capabilități relevante pentru AI-103. Catalogul complet de modele și conectori nu este activat integral. Serviciile cloud rămân în Azure; containerul AI este integrarea .NET. [S01, S39]

Configurarea inițială publică numai web pe 127.0.0.1:8080. Lipsa conturilor de aplicație nu elimină credențialele infrastructurii. Cheile rămân pe server; datele demonstrative sunt seed artificiale. Nu este un portal public pentru date confidențiale.

Limite inițiale

50 MiB/fișier ca limită a aplicației; adaptorul impune limitele serviciului/tier-ului, care pot fi mai mici. Acceptă PDF, imagini și tipurile media configurate; DOCX numai printr-un extractor compatibil. Aliniază Nginx/Kestrel/multipart și verifică MIME magic bytes.

02 / Arhitectura: monolit modular + gateway AI



5 containere locale. Serviciile Foundry rulează în Azure.

Componentă	Responsabilitate și proprietate
Frontend	React/TypeScript compilat cu Vite; Nginx livrează build-ul și face proxy către API.
Backend	Un deploy .NET; deține EF Core, regulile, metadatele, conversațiile și joburile durabile.
AI gateway	Alt deploy .NET; deține doar integrarea Foundry, extragerea, embeddings, indexul și executarea tool-urilor. Nu are DB proprie.
PostgreSQL	Un singur DB al monolitului. La varianta pgvector se adaugă extensia în același container.
MinIO / S3	Stocare de obiecte. Procesarea este în gateway/Foundry; MinIO nu face OCR, transcriere sau inferență.
Azure	Foundry și serviciile integrate rămân cloud; Compose nu le emulează și nu le găzduiește.

Containerul AI este o limită tehnică de integrare cerută de proiect. Backend-ul nu se fragmentează în microservicii; nu apar cozi externe, API gateway separat sau baze de date per modul.

03 / Structura soluției și modulele backend

Modul	Responsabilitate
Requirements	Necesar, criterii, cantitate, monedă și constrângeri.
Offers	Documente ale ofertelor și valori normalizate cu surse.
Orchestration	Stări run/agent step, checkpoint, retry și buget.
Recommendation	Scor calculat în .NET, justificare și raport final.
Documents/Ingestion	Upload S3, hash, extracție, chunks și joburi durabile.
Infrastructure	EF Core, storage, HTTP intern, safety, tracing și health.

Structura minimală a soluției

```
ProcureAI/
  src/
    ProcureAI.Api/                # un monolit modular .NET
      Modules/Requirements/
      Modules/Offers/
      Modules/Orchestration/
      Modules/Recommendation/
      Modules/Documents/
      Modules/Ingestion/
      Infrastructure/Persistence/
      Infrastructure/Storage/
      Program.cs
    ProcureAI.Ai/                 # integrare AI, fără DB proprie
      Providers/Foundry/
      Providers/Extraction/
      Providers/Search/
      Providers/Speech/
      Tools/
      Contracts/
      Program.cs
    procureai-web/          # React + Vite + TypeScript
      src/App.tsx
      src/api.ts
  deploy/
    Api.Dockerfile
    Ai.Dockerfile
    Web.Dockerfile
    Minio.Dockerfile
    nginx.conf
  compose.yaml
  .env.example
  .dockerignore
```

API-ul deține schema, regulile și stările. AI gateway întoarce DTO-uri, nu scrie în PostgreSQL. React apelează numai /api. Nu sunt necesare repository generic, CQRS, Event Sourcing sau MediatR pentru acest nucleu. Modules sunt foldere și servicii în același proiect/deploy.

04 / Stack, versiuni și biblioteci

Componentă	Alegere
Runtime	.NET 10 / ASP.NET Core 10, C# 14. .NET 10 este LTS. [S02]
Persistență	EF Core 10 + Npgsql.EntityFrameworkCore.PostgreSQL compatibil 10.x. [S03, S04]
Frontend	React + Vite + TypeScript; Node 22.12+ pentru toolchain; build static. [S05]
DB	PostgreSQL 17 pentru exemplul Compose; păstrează volumele și fixează versiunea/digestul la deploy.
Object storage	Protocol S3 prin AWSSDK.S3; endpoint MinIO cu ForcePathStyle. [S29]
Integrare de bază	HttpClient + System.Text.Json pentru contractele REST; Azure.Identity pentru Entra.
SDK Foundry nou	Azure.AI.Projects, Azure.AI.Projects.Agents, Azure.AI.Extensions.OpenAI și Azure.Identity. Verifică versiunile compatibile. [S06]
Search / documente	Azure.Search.Documents sau REST; Content Understanding REST 2025-11-01 pentru exemplul stabil. [S09, S10]
Observabilitate	OpenTelemetry.Extensions.Hosting, instrumentare ASP.NET/HttpClient; Azure Monitor exporter când se activează Azure. [S18]

Foundry (new) și Foundry (classic) au SDK-uri și contracte diferite. Nu combina Azure AI Projects 1.x, vechile threads/runs și un exemplu nou de Responses API în același adaptor. Documentația actuală are și API-uri marcate prerelease; fixează package versions și lock files. [S06]

Inițializare proiecte - apoi fixează versiunile restaurate

```
dotnet new sln -n ProcureAI
dotnet new web -n ProcureAI.Api -o src/ProcureAI.Api -f net10.0
dotnet new web -n ProcureAI.Ai -o src/ProcureAI.Ai -f net10.0
dotnet sln add src/ProcureAI.Api/ProcureAI.Api.csproj
dotnet sln add src/ProcureAI.Ai/ProcureAI.Ai.csproj
dotnet add src/ProcureAI.Api package Npgsql.EntityFrameworkCore.PostgreSQL
dotnet add src/ProcureAI.Api package Microsoft.EntityFrameworkCore.Design
dotnet add src/ProcureAI.Api package AWSSDK.S3
dotnet add src/ProcureAI.Ai package Azure.Identity
dotnet add src/ProcureAI.Ai package AWSSDK.S3
npm create vite@latest src/procureai-web -- --template react-ts
```

Selectează EF/Npgsql 10.x compatibile, apoi activează RestorePackagesWithLockFile și păstrează packages.lock.json și package-lock.json în git. Comenzile fără versiune sunt pentru bootstrap, nu pentru un build reproductibil.

05 / Schema PostgreSQL: nucleu comun și domeniu

Tabel comun	Câmpuri și rol
Documents	Id, hash, ObjectKey, FileName, MIME, Size, Status, ActiveGeneration. Original în S3.
DocumentChunks	Id, DocumentId, Generation, Ordinal, Text, PageNumber/StartMs/EndMs.
IngestionJobs	Id, DocumentId?, Kind, TargetId?, PipelineVersion, State, LeaseToken, LeaseUntil, OperationUrl?, Attempt.
Conversations / Messages	Conversație anonimă; mesaje și usage, DomainKind și DomainId ca legătură logică validată.
Citations	MessageId, DocumentId?/ChunkId?, snapshot excerpt, pagină/timestamp și source generation.

Tabel de domeniu	Câmpuri	Relații / reguli
ProcurementRuns	Id, Brief, CriteriaJson, State, Revision, BudgetTokens, SnapshotHash	Un run este o analiză, nu comandă de cumpărare.
RunDocuments	RunId, DocumentId, Role	Ofertă/politică/specificație; unique(RunId, DocumentId).
Offers	Id, RunId, SupplierLabel, Currency?, NetTotal?, Shipping?, LeadDays?, EvidenceJson	Câmpuri cu null-uri; comparația folosește unități și monedă identice.
AgentSteps	Id, RunId, Role, InputHash, PromptVersion, State, OutputJson, UsageJson, Attempt	Unique(RunId, Role, InputHash, PromptVersion); checkpoint durabil.
Recommendations	Id, RunId, SnapshotHash, ScoresJson, ReportJson, CreatedAtUtc	Un rezultat pentru snapshot; toate scorurile se reproduc în .NET.

Integritate și identitate

UUID, timp UTC în timestamptz, JSON în jsonb și decimal în numeric. Configurează explicit ToTable pentru nume predictibile, în schema app; migrația și SQL-ul de lease trebuie să folosească aceleași nume. Revision este application-managed concurrency token: incrementată la fiecare mutație, cu valoarea așteptată în DTO.

Originalele sunt immutable S3 objects; rezultatele raw și rapoartele pot fi stocate în artifacts/{documentId}/{pipelineVersion}/. O ștergere ascunde imediat documentul și apoi curăță index/obiecte. Păstrează snapshot-ul de citare și folosește FK nullable/SetNull când originalul poate fi șters.

06 / Entități și configurare EF Core

Entități principale - fragment C# de referință

```
public enum DocumentStatus { Uploaded, Processing, Ready, Failed, Deleting }

public sealed class Document
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public required string FileName { get; set; }
    public required string ContentType { get; set; }
    public required string Sha256 { get; set; }
    public required string ObjectKey { get; set; }
    public long SizeBytes { get; set; }
    public DocumentStatus Status { get; set; }
    public int ActiveGeneration { get; set; }
    public DateTimeOffset CreatedAtUtc { get; set; } = DateTimeOffset.UtcNow;
    public List<DocumentChunk> Chunks { get; set; } = [];
}

public sealed class DocumentChunk
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public Guid DocumentId { get; set; }
    public Document Document { get; set; } = null!;
    public int Generation { get; set; }
    public int Ordinal { get; set; }
    public required string Text { get; set; }
    public int? PageNumber { get; set; }
    public long? StartMs { get; set; }
    public long? EndMs { get; set; }
}
```

AppDbContext - configurație de bază

```
using Microsoft.EntityFrameworkCore;

public sealed class AppDbContext(DbContextOptions<AppDbContext> options)
    : DbContext(options)
{
    public DbSet<Document> Documents => Set<Document>();
    public DbSet<DocumentChunk> DocumentChunks => Set<DocumentChunk>();

    protected override void OnModelCreating(ModelBuilder model)
    {
        model.HasDefaultSchema("app");
        var documents = model.Entity<Document>();
        documents.HasKey(x => x.Id);
        documents.HasIndex(x => x.Sha256).IsUnique();
        documents.Property(x => x.Sha256).HasMaxLength(64);
        documents.Property(x => x.FileName).HasMaxLength(240);
        documents.Property(x => x.Status).HasConversion<string>();

        var chunks = model.Entity<DocumentChunk>();
        chunks.HasKey(x => x.Id);
        chunks.HasIndex(x => new { x.DocumentId, x.Generation, x.Ordinal })
            .IsUnique();
        chunks.HasOne(x => x.Document).WithMany(x => x.Chunks)
            .HasForeignKey(x => x.DocumentId);
    }
}
```

Completează entitățile pentru Jobs, Conversations, Messages și Citations după tabelul din capitolul 5. Pentru joburi, LeaseToken este concurrency token. Pentru citări păstrează snapshot-ul și folosește FK-uri nullable / DeleteBehavior.SetNull unde sursa poate fi eliminată.

Entități specifice și configurare EF

Fragmente de domeniu; adaugă DbSet și entitățile restante

```
public sealed class ProcurementRun
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public required string Brief { get; set; }
    public required string CriteriaJson { get; set; }
    public required string SnapshotHash { get; set; }
    public string State { get; set; } = "Created";
    public long Revision { get; set; }
    public long BudgetTokens { get; set; } = 15000;
}
public sealed class AgentStep
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public Guid RunId { get; set; }
    public required string Role { get; set; }
    public required string InputHash { get; set; }
    public required string PromptVersion { get; set; }
    public string State { get; set; } = "Pending";
    public string? OutputJson { get; set; }
    public int Attempt { get; set; }
}
// În OnModelCreating; joburi și AgentSteps sunt controlate de API.
model.Entity<ProcurementRun>().Property(x => x.Revision)
    .IsConcurrencyToken();
model.Entity<ProcurementRun>().Property(x => x.CriteriaJson)
    .HasColumnType("jsonb");
model.Entity<AgentStep>().Property(x => x.OutputJson)
    .HasColumnType("jsonb");
model.Entity<AgentStep>().HasIndex(x => new {
    x.RunId, x.Role, x.InputHash, x.PromptVersion }).IsUnique();
model.Entity<AgentStep>().HasOne<ProcurementRun>().WithMany()
    .HasForeignKey(x => x.RunId).OnDelete(DeleteBehavior.Cascade);
```

Liniile de model.Entity sunt corpul OnModelCreating, nu instrucțiuni top-level lângă clase. Completează toate tabelele din cap. 5, FK-urile către Documents și dimensiunile stringurilor. Nu construi un index unic de business pentru valori lipsă fără o regulă explicită.

07 / DbContext, tranzacții și migrații

Înregistrare și modul one-shot pentru migrare

```
using Microsoft.EntityFrameworkCore;

bool migrateOnly = args.Contains("--migrate");
string[] webArgs = args.Where(x => x != "--migrate").ToArray();
var builder = WebApplication.CreateBuilder(webArgs);
string db = builder.Configuration.GetConnectionString("App")
    ?? throw new InvalidOperationException("ConnectionStrings:App missing");

builder.Services.AddDbContextFactory<AppDbContext>(options =>
    options.UseNpgsql(db));
var app = builder.Build();

if (migrateOnly)
{
    await using var scope = app.Services.CreateAsyncScope();
    var factory = scope.ServiceProvider
        .GetRequiredService<IDbContextFactory<AppDbContext>>();
    await using var context = await factory.CreateDbContextAsync();
    await context.Database.MigrateAsync();
    return;
}

// Serviciile/worker-ul se înregistrează înainte de Build.
// Endpoint-urile se mapează după Build și înainte de Run.
app.MapGet("/health/live", () => Results.Ok(new { status = "ok" }));
await app.RunAsync();
```

Pentru fiecare cerere/unitate de lucru și fiecare iterație de worker creezi un DbContext nou. Nu folosi același context în Task.WhenAll pentru operații EF paralele. Păstrează tranzacțiile DB scurte; un apel Foundry de zeci de secunde nu rulează în interiorul unei tranzacții. [S23]

Strategia de migrare

• Migrațiile sunt generate la dezvoltare și incluse în proiectul API.

• Compose rulează migrate o singură dată, înainte să pornească API-ul.

• Nu folosi EnsureCreated împreună cu migrații; verifică snapshot-ul și SQL-ul.

• La deploy-uri controlate poate fi folosit un migration bundle sau SQL idempotent. [S24]

Comenzi pentru schema inițială

```
dotnet new tool-manifest
dotnet tool install dotnet-ef --version 10.*
dotnet ef migrations add InitialSchema --project src/ProcureAI.Api
dotnet ef migrations script --idempotent --project src/ProcureAI.Api
```

Snippet-urile din document sunt unități de implementare. Ai nevoie de toate entitățile, serviciile și înregistrările DI pentru un build complet; niciun fragment scurt nu este prezentat drept aplicație completă.

08 / Contractele API publice și interne

Endpoint de domeniu	Contract
POST /api/procurement/runs	brief + criteria + currency; 201 + id.
POST /api/procurement/runs/{id}/documents	documentId + role; document Ready sau job dependent.
POST /api/procurement/runs/{id}/start	202 + jobId; snapshot imutabil și buget.
GET /api/procurement/runs/{id}	stare, agent steps, usage, recomandare și citări.
POST /api/procurement/runs/{id}/retry	Reia numai pașii eșuați pentru același snapshot.
GET /api/procurement/runs/{id}/report	Raport JSON și matrice de comparație; fără achiziție efectivă.

Endpoint comun	Comportament
POST /api/documents	multipart file; 202 documentId/jobId nou, 200 hash existent.
GET /api/documents/{id}	Metadate/stare; {id}/content pentru stream original.
POST /api/documents/{id}/retry	Replanifică un job de extracție eșuat.
DELETE /api/documents/{id}	202 + cleanup; afectează analizele care citează sursa.
POST /api/chat	Întrebare generală pe sursele permise; contract comun din cap. 23.

Contractul intern pentru un pas AI

```
public sealed record DomainStepRequest(
    Guid CorrelationId, Guid OperationId, string Domain,
    Guid TargetId, string Step, string InputHash,
    string PromptVersion, string InputJson,
    IReadOnlyList<Guid> AllowedDocumentIds);
public sealed record DomainStepResult(
    string Status, string OutputJson, string Deployment,
    long InputTokens, long OutputTokens, long DurationMs);
```

POST /internal/domain/step pentru un pas de domeniu; /internal/extractions/start și /poll; /internal/embeddings, /index/upsert, /index/delete, /retrieve și /generate. Gateway-ul primește ObjectKey numai din backend. Domain/Step sunt allowlisted, nu prompturi arbitrare trimise din browser.

Backend-ul injectează source scope; un conversationId anonim nu oferă autorizare. Folosește requestId pentru deduplicarea unei cereri, ProblemDetails cu code/traceId și 409 pentru revision/stare incompatibilă. Cererile lungi întorc 202 și folosesc polling pe job/resource.

09 / Upload și stocarea fișierelor în S3

Upload-ul intră prin backend. Validezi mărimea, extensia și tipul real al fișierului; faci streaming/spooling într-un fișier temporar, calculezi SHA-256 și generezi ObjectKey. După stocarea în S3, salvezi Document + IngestionJob în aceeași tranzacție PostgreSQL.

Ordinea operațiilor și compensarea

• Spool temporar limitat ca dimensiune; nu folosi MemoryStream pentru video mare.

• Caută Sha256 existent; dacă există, reutilizează documentul și șterge temporarul.

• Încarcă originalul în S3, apoi commit Document + job; dacă DB eșuează, încearcă ștergerea obiectului.

• Un obiect rămas orfan este curățat prin reconciliere periodică; DB și S3 nu au tranzacție comună.

• Unique index pe hash rezolvă upload-urile concurente. Tratarea conflictului nu trebuie să dubleze joburile.

Client S3 pentru MinIO - înregistrare DI

```
using Amazon.Runtime;
using Amazon.S3;

builder.Services.AddSingleton<IAmazonS3>(_ =>
    new AmazonS3Client(
        new BasicAWSCredentials(
            builder.Configuration["Storage:AccessKey"]!,
            builder.Configuration["Storage:SecretKey"]!),
        new AmazonS3Config
        {
            ServiceURL = builder.Configuration["Storage:Endpoint"],
            ForcePathStyle = true,
            AuthenticationRegion = "us-east-1"
        }));
```

Scrierea unui obiect - după validare și calcularea hash-ului

```
using Amazon.S3;
using Amazon.S3.Model;

public sealed class ObjectStore(IAmazonS3 s3, IConfiguration config)
{
    private readonly string bucket = config["Storage:Bucket"]!;

    public async Task PutAsync(string key, Stream input, string contentType,
        CancellationToken ct)
    {
        await s3.PutObjectAsync(new PutObjectRequest
        {
            BucketName = bucket,
            Key = key,
            InputStream = input,
            ContentType = contentType,
            AutoCloseStream = false
        }, ct);
    }

    public Task<GetObjectResponse> OpenAsync(string key, CancellationToken ct)
        => s3.GetObjectAsync(new GetObjectRequest
        {
            BucketName = bucket, Key = key
        }, ct);
}
```

La bootstrap, verifică și creează bucket-ul prin API S3, cu retry limitat. La download păstrează GetObjectResponse deschis până se termină streaming-ul și dispune-l la final. Nu expune endpoint-ul intern minio:9000 în citările browserului. [S29]

10 / Joburi durabile fără Redis sau RabbitMQ

BackgroundService rulează în containerul backend, dar starea joburilor este în PostgreSQL. Gateway-ul nu ține coada în memorie. Un restart al API-ului nu pierde fișierul și nu resetează progresul. [S25]

Pas	Rezultat durabil
StartExtraction	ProviderOperation este persistat imediat după răspunsul 202 al providerului.
PollExtraction	Dacă analiza continuă, jobul este replanificat; polling-ul nu crește Attempt.
Normalize	normalized.json în S3; chunk-uri cu generație nouă în DB.
EmbedAndIndex	Batch-uri limitate; upsert cu chei deterministe și verificarea rezultatului fiecărui item.
Activate	Document devine Ready și ActiveGeneration se schimbă numai după indexare completă.
Cleanup	Șterge vectorii/chunk-urile vechi și obiectele după politica aleasă.

Claim atomic - SQL adaptat tabelelor EF cu nume quoted

```
UPDATE app."IngestionJobs"
SET "Status" = 'Running',
    "LeaseToken" = @token,
    "LeaseUntilUtc" = now() + interval '90 seconds'
WHERE "Id" = (
    SELECT "Id" FROM app."IngestionJobs"
    WHERE ("Status" IN ('Queued', 'RetryWaiting')
           AND "NextAttemptAtUtc" <= now())
       OR ("Status" = 'Running' AND "LeaseUntilUtc" < now())
    ORDER BY "NextAttemptAtUtc", "Id"
    FOR UPDATE SKIP LOCKED
    LIMIT 1
)
RETURNING *;
```

Execuți claim-ul ca o singură comandă parametrizată. Commit imediat; nicio tranzacție nu rămâne deschisă în timpul apelului AI. Update-ul final are condiția Id + LeaseToken. Dacă lease-ul se pierde, worker-ul vechi nu poate activa generația. Reînnoiește lease-ul pentru pași lungi sau păstrează fiecare apel sub timpul lease-ului.

Retry și idempotency

• 429/5xx/timeout: backoff cu jitter, respectă Retry-After; maximum 3 încercări inițial.

• 400 pentru fișier invalid, 401/403 și deployment inexistent: eroare explicabilă, nu retry repetat.

• Cheia chunk-ului/indexului derivă din document + generație + ordinal; upsert este repetabil.

• Un apel cloud reușit urmat de crash înainte de persistare poate fi repetat și facturat din nou. Nu promite exactly-once pentru Foundry.

• Operațiile de generare media sunt mai costisitoare: persistă operation/job ID și reia polling-ul.

Extinde IngestionJobs cu Kind, TargetId și PipelineVersion pentru fluxul domeniului. Un job de document poate exista independent de un job de analiză. Un TargetId de domeniu nu trebuie salvat în DocumentId; FK-ul pentru document rămâne nullable în joburile de analiză.

11 / Extragere și normalizare pentru acest proiect

PDF-urile ofertelor și fișele tehnice se normalizează în câmpuri și evidență. O fotografie/înregistrare este sursă suplimentară, nu cotație comercială. Normalizează unitățile și baza prețului înainte de orice scor; un preț fără cantitate sau monedă nu devine zero.

Modalitate	Rezultat intern	Regulă
PDF / scan / imagine	Text, layout, câmpuri și evidence regions	Sursa exactă și hash păstrate.
Text / DOCX acceptat	Paragrafe, headings și spans	Fără executarea instrucțiunilor în conținut.
Audio	Transcript și segment timestamps	Optional; în același gateway, prin Speech/CU.
Video	Scene, cadre/descrieri și transcript	Optional; proveniență temporală și limite de durată.

DTO normalizat comun

```
public sealed record EvidenceSource(Guid DocumentId, string SourceId,
    int? PageNumber, long? StartMs, long? EndMs,
    string Text, string? RegionJson);
public sealed record NormalizedExtraction(Guid DocumentId,
    string Provider, string ModelVersion, string RawObjectKey,
    IReadOnlyList<EvidenceSource> Sources, string FieldsJson);
```

Chunking și filtrarea surselor

RAG caută cerințe, politici și specificații în documentele acestui run. Filtrul RunId/DocumentIds vine din backend. Pentru comparația numerică folosește Offers normalizate. Agentic retrieval poate planifica subîntrebări tehnice, dar rezultatul final citează numai dovezi validate.

Păstrează un SourceId/ChunkId stabil pentru fiecare generație. Ingestion nu face Ready înainte ca raw result, chunks și indexul să fie disponibile. Pentru reindexare, construiește generația nouă și schimbă ActiveGeneration numai după succes; rezultatele vechi nu sunt surse active.

Testul de referință

Necesar seed: 10 unități, monedă EUR, livrare ≤10 zile. Oferta A are net 1000 + transport 50, 7 zile; B are net 980 + transport 120, 9 zile. costul comparabil este 1050 vs 1100; .NET stabilește scorurile și modelul explică alegerea.

12 / Strategii RAG interschimbabile

Strategie	Unde sunt datele	Când o alegi
Classic hybrid RAG - recomandat	Chunk-uri în DB; vectori/text în Azure AI Search; răspuns în Foundry.	Control bun al surselor, filtrare, debugging și apropiere de AI-103.
pgvector RAG	Chunk-uri + vectori în PostgreSQL; inferență și embeddings tot în Foundry.	Un container DB existent, flexibilitate locală; construiești ranking-ul și evaluarea.
Foundry File Search	Fișiere încărcate suplimentar în stocarea/vector store-ul serviciului.	Prototip de agent; verifică tipuri acceptate, cost și sincronizarea cu MinIO.
Agentic retrieval / Foundry IQ	Knowledge sources și retrieval în Azure AI Search/Foundry.	Întrebări complexe; mai multă orchestrare și cost, cu API-uri GA/preview distincte.
Fără RAG	Un document mic trimis direct modelului, în limita de context.	Diagnostic și comparație; nu este motorul unei biblioteci mari.

Interfața providerului - în gateway

```
public sealed record RetrievalHit(
    Guid ChunkId, Guid DocumentId, int Generation,
    string Text, double Score,
    int? PageNumber, long? StartMs, long? EndMs);

public interface IRetrievalProvider
{
    Task<IReadOnlyList<RetrievalHit>> SearchAsync(
        string query, IReadOnlyList<Guid>? documentIds,
        int take, CancellationToken ct);

    Task UpsertAsync(IReadOnlyList<IndexChunk> chunks, CancellationToken ct);
    Task DeleteDocumentAsync(Guid documentId, CancellationToken ct);
}
```

IndexChunk este DTO-ul text + metadata + embedding al providerului. Backend-ul selectează numai surse Ready și validează generația înainte de generare. Începe cu un singur provider activ, configurat prin Rag:Provider; schimbarea providerului necesită reindexare și reevaluare.

Agentic retrieval are capabilități GA și preview. Documentația actuală indică 2026-04-01 pentru retrieval minimal cu surse GA și 2026-08-01-preview pentru query planning/synthesis și alte extensii. Nu descrie întreaga familie ca exclusiv GA sau exclusiv preview. [S12]

Varianta fără RAG

Compose din cap. 25 este profilul complet cu AzureSearch. Pentru NoRag setează RAG_PROVIDER=NoRag, omite Search__Endpoint/ApiKey/Index și embedding deployment din serviciul ai și nu activează pașii de index/retrieval. Validarea options/readiness se face doar pentru providerul ales. Conținutul structurat al documentului se furnizează direct în limitele de context; pipeline-ul nu pretinde retrieval dacă nu îl execută.

13 / Indexul Azure AI Search

Indexul conține un document per chunk. Modelul de embedding și dimensiunea vectorului sunt aceleași la ingestion și query. Folosește identificatori proprii, nu numele fișierului drept key. Indexarea este făcută explicit de aplicație pentru a păstra MinIO ca sursă de obiecte. [S09, S13]

Schema de referință pentru indexul procureai-chunks

```
{
  "name": "procureai-chunks",
  "fields": [
    {"name":"id","type":"Edm.String","key":true},
    {"name":"documentId","type":"Edm.String","filterable":true},
    {"name":"generation","type":"Edm.Int32","filterable":true},
    {"name":"title","type":"Edm.String","searchable":true},
    {"name":"content","type":"Edm.String","searchable":true},
    {"name":"pageNumber","type":"Edm.Int32"},
    {"name":"startMs","type":"Edm.Int64"},
    {"name":"endMs","type":"Edm.Int64"},
    {
      "name":"contentVector",
      "type":"Collection(Edm.Single)",
      "searchable":true,
      "retrievable":false,
      "dimensions":1536,
      "vectorSearchProfile":"procureai-vector"
    }
  ],
  "vectorSearch": {
    "algorithms": [{
      "name":"procureai-hnsw",
      "kind":"hnsw",
      "hnswParameters":{"metric":"cosine"}
    }],
    "profiles": [{
      "name":"procureai-vector",
      "algorithm":"procureai-hnsw"
    }]
  },
  "semantic": {
    "configurations": [{
      "name":"procureai-semantic",
      "prioritizedFields": {
        "titleField":{"fieldName":"title"},
        "prioritizedContentFields":[{"fieldName":"content"}],
        "prioritizedKeywordsFields":[]
      }
    }]
  }
}
```

1536 este dimensiunea aleasă pentru text-embedding-3-small. Dacă o reduci sau schimbi modelul, recreează câmpul/indexul și reindexează corpusul. Nu amesteca vectori incompatibili. Pentru semantic ranking verifică activarea și costurile tier-ului ales. JSON-ul se trimite la API-ul Search corespunzător sau se transpune în Azure.Search.Documents.

14 / Retrieval, ranking și validarea surselor

Cerere hybrid - vectorul este produs de modelul de embeddings

```
{
  "search": "Cum se configurează retry pentru Service Bus?",
  "vectorQueries": [{
    "kind":"vector",
    "vector":[0.013, -0.008, 0.021],
    "fields":"contentVector",
    "k":50
  }],
  "queryType":"semantic",
  "semanticConfiguration":"procureai-semantic",
  "select":"id,documentId,generation,title,content,pageNumber,startMs,endMs",
  "top":12
}
```

Vectorul de trei valori de mai sus ilustrează forma cererii; request-ul real trebuie să conțină toate cele 1536 valori. k=50 este punctul de pornire pentru candidații vectoriali; top=12 este setul returnat. Măsoară rezultatele înainte de a fixa aceste valori.

Fluxul recomandat pentru întrebări

• Backend-ul validează întrebarea, citește istoricul limitat și creează CorrelationId.

• Gateway-ul execută retrieval și întoarce candidații, fără să genereze încă răspunsul.

• Backend-ul verifică Document.Status=Ready și ActiveGeneration pentru fiecare hit. Elimină sursele șterse sau vechi.

• Alege aproximativ 4-8 surse în limita de tokeni, cu diversitate; construiește etichetele S1, S2 etc.

• Trimite numai sursele validate către gateway pentru generare. Dacă nu există dovezi, răspunsul explică lipsa informației.

• Validează citările din răspuns, salvează mesajul și usage, întoarce sursele către frontend.

Nu interpreta scorul Search drept probabilitate sau confidence universală. Pragurile se calibrează cu un dataset. Un scor mic poate însemna întrebare nerelevantă, ingestion slab sau terminologie diferită; verifică și sănătatea indexului.

Filtrare și ștergere

DocumentIds este opțional. Filtrul se construiește din Guid-uri validate, nu din expresii OData trimise de browser. Pentru corpusuri mari, folosește versiuni/generații și un mecanism de publicare coerent; backend-ul este gardianul final al surselor. După DELETE, sursa nu mai intră în prompt chiar dacă eliminarea fizică a indexului este în curs.

Scope-ul domeniului este rezolvat de backend în AllowedDocumentIds. RunId/InvoiceId/VersionId nu sunt câmpuri ale indexului comun de mai sus: API-ul le mapează la documente permise și construiește filtrul documentId. Dacă adaugi câmpuri de scope în index, actualizează schema, upload-ul și filtrul împreună.

15 / Prompturi, JSON și validarea rezultatului

Rolul tău este limitat la etapa primită. Folosește snapshot-ul și tools permise. Nu inventa furnizori, cotații, termene sau certificări. Nu schimba ponderile și scorurile calculate de .NET. Dacă ofertele nu sunt comparabile, întoarce needs_input cu câmpurile lipsă. Nu plasa comenzi și nu contacta furnizori.

Format de ieșire intern - exemplu JSON

```
{
  "status": "completed",
  "recommendedOfferId": "offer-A",
  "scoreSnapshotId": "score-v1",
  "reasons": [
    {
      "text": "Cost comparabil și termen în limită",
      "citationLabels": [
        "S1",
        "S2"
      ]
    }
  ],
  "unknowns": []
}
```

Validare înainte de persistare

• Parsează JSON și validează schema/enum/limitele. Retrimite maximum o dată pentru repararea formatului; apoi marchează eroare sigură.

• Toate evidence/citation IDs trebuie să fie în setul permis al request-ului și în generația activă.

• Backend-ul construiește URL-urile citărilor; modelul nu produce adrese arbitrare.

• Model/deployment, promptVersion, inputHash, usage și duration sunt atașate rezultatului.

• Conținutul extras și rezultatele tools sunt date neîncrezătoare; nu pot suprascrie instrucțiunile serverului.

Exemplul JSON este un format al aplicației. Folosește Structured Outputs numai dacă deployment-ul și API-ul ales le suportă; păstrează validatorul backend și la acel mod. Pentru Responses API colectează textul din output/message/content, verifică status și tratează explicit incomplete sau function_call. [S07-S08]

16 / AI Gateway .NET: contract REST și autentificare

Un proiect ASP.NET Core separat expune numai endpoint-uri /internal. Nucleul folosește REST prin HttpClient pentru a evita amestecarea SDK-urilor new/classic. Endpoint-ul de inferență v1 al resursei și endpoint-ul de proiect au roluri distincte. [S06, S07, S08]

Endpoint	Utilizare
https://RESOURCE.openai.azure.com/openai/v1/	Responses și embeddings pentru deployment-urile Azure OpenAI. Exemplul de laborator folosește api-key.
https://RESOURCE.services.ai.azure.com/api/projects/PROJECT	Agenți, conversații și funcții de proiect Foundry; pentru exemplele SDK folosește Entra și rolurile potrivite.
https://RESOURCE.cognitiveservices.azure.com/	Endpointul configurat pentru Content Understanding; verifică valoarea returnată de resource.

Clientul v1 - implementare proprie, fără retry automat în acest fragment

```
using System.Net.Http.Json;
using System.Text.Json;

public sealed class FoundryV1Client(HttpClient http, IConfiguration config)
{
    public async Task<JsonDocument> PostAsync(
        string path, object payload, CancellationToken ct)
    {
        var root = new Uri(config["Foundry:OpenAiEndpoint"]!);
        using var request = new HttpRequestMessage(HttpMethod.Post,
            new Uri(root, path));
        request.Headers.Add("api-key", config["Foundry:ApiKey"]!);
        request.Content = JsonContent.Create(payload);

        using var response = await http.SendAsync(request, ct);
        response.EnsureSuccessStatusCode();
        await using var body = await response.Content.ReadAsStreamAsync(ct);
        return await JsonDocument.ParseAsync(body, cancellationToken: ct);
    }
}
```

Inferență Responses - payload adaptat aplicației

```
using var result = await foundry.PostAsync("responses", new
{
    model = config["Foundry:ChatDeployment"],
    instructions = systemPrompt,
    input = groundedQuestionWithSources,
    max_output_tokens = 1200,
    store = false
}, cancellationToken);

var textParts = result.RootElement.GetProperty("output")
    .EnumerateArray()
    .Where(item => item.GetProperty("type").GetString() == "message")
    .SelectMany(item => item.GetProperty("content").EnumerateArray())
    .Where(part => part.GetProperty("type").GetString() == "output_text")
    .Select(part => part.GetProperty("text").GetString());
string answer = string.Concat(textParts);
```

17 / Embeddings și varianta pgvector

Batch embeddings - ordonează răspunsul după index

```
using var result = await foundry.PostAsync("embeddings", new
{
    model = config["Foundry:EmbeddingDeployment"],
    input = chunks.Select(x => x.Text).ToArray(),
    encoding_format = "float"
}, cancellationToken);

float[][] vectors = result.RootElement.GetProperty("data")
    .EnumerateArray()
    .OrderBy(item => item.GetProperty("index").GetInt32())
    .Select(item => item.GetProperty("embedding").EnumerateArray()
        .Select(number => number.GetSingle()).ToArray())
    .ToArray();

if (vectors.Length != chunks.Count || vectors.Any(x => x.Length != 1536))
    throw new InvalidOperationException("Embedding contract mismatch");
```

Validează tokenii pe chunk și pe batch, dimensiunea vectorului și cota deployment-ului. Cache-ul natural este hash(text normalizat + model/version + dimensiune); poți evita embeddings repetate pentru chunk-uri identice. model este numele deployment-ului Azure, nu obligatoriu ID-ul de catalog. [S13]

Alternativa locală în același PostgreSQL

Pentru pgvector instalează extensia în imaginea PostgreSQL aleasă sau folosește o imagine pgvector verificată pentru aceeași versiune majoră. În .NET folosește Pgvector.EntityFrameworkCore și configurarea UseVector potrivită versiunilor fixate. Nu este nevoie de încă un DB. [S27, S28]

În această variantă, interogarea EF/pgvector se execută în modulul Knowledge al backend-ului, care deține DB. Gateway-ul produce embeddings; un adaptor HTTP deleagă retrieval-ul local către backend. Nu îi dai acces direct la DB. RAG_PROVIDER=PgVector singur nu este suficient: adaptează imaginea DB, migrațiile și fă configurația Search opțională în Compose.

Schemă vectorială - variantă SQL opțională

```
CREATE EXTENSION IF NOT EXISTS vector;
ALTER TABLE app."DocumentChunks"
ADD COLUMN "Embedding" vector(1536);

CREATE INDEX ix_chunks_embedding
ON app."DocumentChunks"
USING hnsw ("Embedding" vector_cosine_ops);

SELECT "Id", "DocumentId", "Text"
FROM app."DocumentChunks"
WHERE "Embedding" IS NOT NULL
ORDER BY "Embedding" <=> @query_vector
LIMIT 12;
```

@query_vector este parametru vector tipizat, nu string interpolat. Pentru hybrid retrieval local adaugi full-text search și combini listele, de exemplu prin reciprocal rank fusion. pgvector nu furnizează automat semantic reranking de Azure AI Search. La filtre restrictive evaluează recall-ul și configurația HNSW.

18 / Content Understanding: fișiere binare și operații async

Foundry nu poate accesa http://minio:9000 sau localhost din rețeaua ta Compose. Gateway-ul citește obiectul S3 și trimite bytes către analyzeBinary. Pentru fișiere mari și operații care cer URL-uri public accesibile poți folosi temporar Azure Blob cu SAS limitat; MinIO rămâne sursa canonicală. Nu expune MinIO anonim.

Pornirea extragerii - REST 2025-11-01

```
using var inputObject = await storage.OpenAsync(objectKey, ct);
using var request = new HttpRequestMessage(HttpMethod.Post,
    $"{endpoint.TrimEnd('/')}/contentunderstanding/analyzers/" +
    $"{Uri.EscapeDataString(analyzerId)}:analyzeBinary?api-version=2025-11-01");
request.Headers.Add("Ocp-Apim-Subscription-Key", contentUnderstandingKey);
request.Content = new StreamContent(inputObject.ResponseStream);
request.Content.Headers.ContentType =
    new System.Net.Http.Headers.MediaTypeHeaderValue("application/octet-stream");

using var response = await http.SendAsync(request, ct);
response.EnsureSuccessStatusCode();
string operationUrl = response.Headers.GetValues("Operation-Location").Single();
// Întoarce URL-ul către backend pentru persistare în IngestionJob.
```

Răspunsul 202 confirmă acceptarea, nu finalizarea. Backend-ul persistă ProviderOperation înainte de a continua. Polling-ul trece prin gateway, verifică host-ul din URL și reatașează autentificarea. Nu presupune că operația este gata după o întârziere fixă. [S10]

Selectarea analyzer-ului

Modalitate	Punct de pornire
Documente pentru RAG	prebuilt-documentSearch; păstrezi markdown/layout și pozițiile.
Audio pentru cunoștințe	prebuilt-audioSearch sau adaptor Speech, în funcție de rezultat.
Video pentru cunoștințe	prebuilt-videoSearch pentru segmente și transcript.
Imagini	Analyzer imagine de bază/custom pentru schema dorită.
Câmpuri specifice	Analyzer custom cu fieldSchema și proveniență; testat pe set reprezentativ.

Verifică ID-urile efectiv disponibile în versiunea API și în resursă; creează/configurează analyzer-ul când este necesar. Content Understanding are extracție multimodală GA și agentic document understanding în API-ul preview 2026-06-01. Ghidul AI-103 încă menționează „pro-mode”; tratează separat contractele din documentația actuală. [S01, S11, S30]

19 / Agenți Foundry și tools specifice domeniului

Tool	Tip	Limita de execuție
search_supplier_evidence	Read	Fixture/documente; web search doar dacă activat și cu surse.
extract_offer	Read	CU/Document Intelligence în funcție de document; evidență.
validate_requirements	Read	Reguli .NET pentru cantitate/unități/criterii.
score_offers	Read	Scor .NET determinist, fără execuție de cod arbitrar.
build_recommendation	Draft	Raport cu score IDs și evidence IDs; fără comandă/email.

Bucla de execuție

Un function tool este executat în aplicație după ce modelul îl cere; declararea JSON-ului nu implementează funcția. Gateway-ul validează numele/argumentele și cere backend-ului operațiile care dețin DB. Backend-ul verifică scope-ul și starea; modelul primește un DTO minim. [S14, S39]

Exemplu function tool pentru Responses API

```
{
  "type": "function",
  "name": "search_supplier_evidence",
  "description": "Citește dovezi strict în scope-ul curent.",
  "parameters": {
    "type": "object",
    "properties": {
      "query": {
        "type": "string"
      }
    },
    "required": [
      "query"
    ],
    "additionalProperties": false
  },
  "strict": true
}
```

Schema este un adaptor de exemplu: completează argumentele reale după tabel; TargetId/allowed sources sunt injectate de server. Pentru function_call din Responses întoarce function_call_output cu același call_id și continuă conversația până la răspuns sau limită. Nu amesteca schema Responses cu funcțiile Chat Completions sau cu lifecycle-ul Runs din SDK classic.

Limite de laborator: maximum 5 tool rounds/cerere, deadline explicit și buget de tokens. Tools cu efecte produc o propunere sau snapshot local. Secretele de infrastructură nu sunt argumente ale modelului.

20 / Toolbox, MCP, OpenAPI și tool-uri găzduite

Capabilitate	Aplicare în ProcureAI	Condiție de integrare
Custom functions	search/open/translate/calculate pentru sursele locale.	Function calling executat în gateway; nu cere acces inbound din cloud la Docker.
OpenAPI	Descrie contractele de knowledge și media processing.	Un tool executat de Foundry trebuie să poată accesa endpoint-ul HTTPS și să se autentifice.
MCP	Expune aceleași tool-uri printr-un server MCP, în aplicația AI sau backend.	Serverul local nu devine automat accesibil Foundry; client local sau endpoint securizat accesibil cloud.
File Search	Variantă de RAG administrat pentru fișiere acceptate.	Upload suplimentar în serviciu; sincronizare/deletion și cost separat.
Code Interpreter	Calcule și grafice din tabele/documente.	Fișierele/dataset-urile se încarcă explicit; sandbox-ul cloud nu vede volumele locale.
Web search	Actualizare documentație cu surse externe, numai când este activată.	Răspunsul distinge surse interne de web; filtre și reguli de confidențialitate.
Toolbox	Gruparea configurațiilor și tool-urilor pentru agenți.	Separă versionarea și autentificarea de logica de business.

Acestea sunt familii de tool-uri documentate; combinația disponibilă depinde de model, tipul agentului și regiune. Modul local de function calling este alegerea simplă de început. MCP/OpenAPI sunt adaptoare suplimentare pentru același contract, nu încă un domeniu sau încă un container obligatoriu. [S15, S16]

Rețea și identitate

Din Foundry, numele api, ai și minio nu se rezolvă ca servicii Compose. Un URL localhost indică mediul cloud care execută tool-ul, nu calculatorul tău. Pentru tool-uri găzduite în cloud folosește conectivitate reală, identitate și endpoint permis; păstrează în laborator function calling cu execuție locală.

Controlul tool-urilor

Tool-urile nucleului sunt read-only. Ștergerea/reindexarea fișierelor se face prin API-ul explicit, nu prin comenzi deduse dintr-un document. Pentru exercițiul AI-103 de aprobare, un tool de generare/reindexare poate întoarce RequiresApproval și este reluat numai după confirmarea cererii; nu se adaugă conturi sau roluri de utilizator.

SDK .NET pentru agent

AIProjectClient + DefaultAzureCredential pornesc integrarea nouă. AgentAdministrationClient versionază agentul; ProjectOpenAIClient oferă Responses pentru model/agent. Publicarea este bootstrap/CI, nu per întrebare. Fixează versiunile compatibile din quickstart. [S06]

21 / Modele, memorie, multi-agent și optimizare

Rol	Model / strategie propusă
Răspuns uzual	procureai-chat: GPT-4.1-mini; compară în evaluări cu un deployment GPT-5-mini.
Reasoning / verificare	Deployment separat pentru sarcini complexe; activat numai când este util.
Embeddings	procureai-embed: text-embedding-3-small, 1536 dimensiuni în această specificație.
Analiză vizuală	Model multimodal disponibil în Foundry + Content Understanding.
Clasificare simplă	SLM precum Phi-4-mini-instruct; verifică schema și limba corpusului.
Generare imagini/video	Deployment GPT Image și Sora 2 unde există acces; API și cost separate.

Memorie fără utilizatori

ConversationId identifică o conversație anonimă. PostgreSQL este sursa de adevăr în modul stateless; trimite ultimele mesaje relevante în limita contextului. Un sumar comprimat poate fi păstrat separat în conversație. Memoria nu este permisiune și nu justifică expunerea datelor public. Modul cu conversație Foundry este alternativ, cu retenție și deletion documentate.

Exercițiu multi-agent în același gateway

Retriever găsește dovezi; Writer redactează; Verifier verifică citările și inconsistențele. Orchestrarea este secvențială și bounded. Poți folosi un model pentru toate rolurile sau modele diferite; nu sunt necesare trei containere. Dacă Verifier cere completări, permite o singură iterație, cu aceleași tool-uri controlate.

Reflection și evaluare

Auto-verificarea primește răspunsul și dovezile, apoi întoarce erori concrete/JSON. Ea nu garantează adevărul. Măsoară diferența față de pipeline-ul simplu și urmărește tokenii/latenta suplimentară. Combină reguli deterministe pentru ID-uri/citări cu verificarea modelului.

Fine-tuning - extensie, nu înlocuitor al RAG

Poate îmbunătăți formatul sau comportamentul specific domeniului pe un model compatibil. Datele actualizabile ale companiei rămân în retrieval. Separă train/test, elimină datele sensibile și verifică deployment/region support. Nu activezi fine-tuning pentru a porni aplicația.

Reutilizează embeddings, limitează contextul și începe cu un model mic și un worker. Azure AI Search poate avea cost fix chiar cu puține întrebări; pgvector îl înlocuiește, dar mută responsabilitatea de retrieval în aplicație.

22 / Imagini, video, sunet și text: adaptoare Foundry

Toate modalitățile folosesc aceeași structură de obiecte S3 și aceleași joburi. Se păstrează distincția dintre **understanding** - informație extrasă din surse - și **generation** - conținut creat. Un clip sau o imagine generată nu devine dovadă pentru afirmații din documentația companiei.

Adaptor	Contract și utilizare minimă
Speech	Transcrie audio pentru ingestion; sintetizează un răspuns în audio. Înregistrează model/voice și limba. Custom Speech este un laborator pentru vocabular tehnic.
Translator	Traduce excerpt-uri/răspunsuri; păstrează sursa originală și limba. Nu înlocuiește automat textul original din index.
Language	Entități, key phrases și PII pentru conținutul ingested. Sentiment/tone se pot testa pe același text fără ecran separat.
Image generation/edit	Generează o ilustrație explicativă sau editează o imagine cu prompt și mască; salvează rezultatul în generated/. Validează regulile de brand și marcajele.
Video generation/remix	Creează un clip explicativ pentru o cerere explicită; persistă ID-ul jobului, face polling și descarcă rezultatul în S3. Sora 2 este preview.
Visual understanding	Descrieri de imagini și regiuni, obiecte/componente, alt-text și întrebări bazate pe dovezi vizuale.
Audio/video understanding	Transcrieri, segmente și capitole; răspunsurile includ timestamp-urile folosite.

Speech, Translator, Content Understanding și modelele pentru generare au endpoint-uri/credite/limite proprii. Adaptoarele păstrează aceeași interfață internă, dar nu trimit toate operațiile către /responses. Consultă API-ul exact al fiecărui serviciu. [S19, S20, S21, S30]

Contract de operație media - intern aplicației

```
public sealed record MediaOperationRequest(
    Guid OperationId,
    string Kind,
    string? SourceObjectKey,
    string? Prompt,
    string? Language,
    string? MaskObjectKey);

public sealed record MediaOperationResult(
    string Status,
    string? ProviderOperation,
    string? ResultObjectKey,
    IReadOnlyList<string> Warnings);
```

Generarea media se activează ca extensie a aceluiași chat/flux de fișiere sau prin client CLI de laborator; nu presupune pagini de marketing, editor complet sau management de asset-uri. API-ul verifică feature flags și consimțământul explicit al cererii înainte să creeze jobul cu cost.

23 / Frontend React + Vite: un singur ecran

Zonă	Comportament
Fișiere	Upload, listă cu stare și retry/delete. Selectarea surselor pentru întrebare.
Conversație	Question input, răspuns, indicator de lucru; streaming este extensie.
Surse	Fișier, pagină/timestamp, excerpt și buton de deschidere original.

Folosește TypeScript și fetch. Nu este necesar Redux sau router pentru un singur ecran. Păstrează ConversationId în memoria paginii sau localStorage; nu este token de autentificare. Randează textul în siguranță; nu executa HTML întors de model.

src/api.ts - client simplu

```
export type Citation = {
  label: string;
  documentId: string;
  chunkId: string;
  fileName: string;
  pageNumber: number | null;
  startMs: number | null;
  endMs: number | null;
  excerpt: string;
  contentUrl: string;
};

export type Answer = {
  conversationId: string;
  answer: string;
  citations: Citation[];
  inputTokens: number;
  outputTokens: number;
  durationMs: number;
};

export async function upload(file: File) {
  const body = new FormData();
  body.append("file", file);
  const response = await fetch("/api/documents", { method: "POST", body });
  if (!response.ok) throw new Error(`Upload failed: ${response.status}`);
  return response.json();
}

export async function ask(question: string, conversationId?: string,
  signal?: AbortSignal): Promise<Answer> {
  const response = await fetch("/api/chat", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ question, conversationId: conversationId ?? null }),
    signal
  });
  if (!response.ok) throw new Error(`AI request failed: ${response.status}`);
  return response.json();
}
```

src/App.tsx - nucleul interacțiunii

```
import { useState } from "react";
import { ask, upload, type Answer } from "./api";

export default function App() {
  const [question, setQuestion] = useState("");
  const [result, setResult] = useState<Answer>();
  const [busy, setBusy] = useState(false);
  const [error, setError] = useState("");

  async function submit() {
    if (!question.trim() || busy) return;
    setBusy(true); setError("");
    try { setResult(await ask(question, result?.conversationId)); }
    catch (e) { setError(e instanceof Error ? e.message : "Request failed"); }
    finally { setBusy(false); }
  }

  return <main>
    <h1>ProcureAI</h1>
    <input type="file" disabled={busy} onChange={async e => {
      const file = e.target.files?.[0];
      if (!file) return;
      setBusy(true); setError("");
      try { await upload(file); }
      catch (err) { setError(err instanceof Error ? err.message : "Upload failed"); }
      finally { setBusy(false); }
    }} />
    <textarea value={question} onChange={e => setQuestion(e.target.value)} />
    <button disabled={busy} onClick={submit}>
      {busy ? "Procesează..." : "Întreabă"}
    </button>
    {error && <p role="alert">{error}</p>}
    {result && <section>
      <p style={{ whiteSpace: "pre-wrap" }}>{result.answer}</p>
      {result.citations.map(source => <article key={source.label}>
        <a href={source.contentUrl}>{source.label}: {source.fileName}</a>
        <p>{source.excerpt}</p>
      </article>)}
    </section>}
  </main>;
}
```

Completează componenta de listă/polling pentru documente și progres. La schimbarea paginii anulează cererile prin AbortController. Citările sunt date de backend; pentru pagina PDF sau timestamp adaugă fragmentul/query potrivit viewer-ului, nu presupune că toate browserele interpretează aceleași fragmente.

Dezvoltare locală

Vite dev server poate face proxy /api la backend; serverul se leagă la 0.0.0.0 în containerul de dev. În imaginea finală rulează Nginx, nu vite preview. Variabilele VITE_* sunt publice în bundle: nu pune chei Azure sau S3 acolo. [S05]

Panoul specific acestei teme

Componenta de chat de mai sus este comună. Adaugă în același App ecranul de domeniu și starea resourceId/jobId, fără router sau conturi. Fluxul concret și clientul TypeScript sunt în cap. 35. Upload-ul comun produce DocumentId; apoi API-ul de domeniu atașează documentul.

24 / Dockerfile-uri pentru cele patru aplicații

PostgreSQL folosește imagine upstream; API, AI, frontend și MinIO au build-uri separate. Exemplele folosesc tag-uri majore pentru lizibilitate; pentru reproductibilitate fixează patch/digest în proiect și actualizează controlat.

deploy/Api.Dockerfile

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY src/ProcureAI.Api/ src/ProcureAI.Api/
RUN dotnet restore src/ProcureAI.Api/ProcureAI.Api.csproj
RUN dotnet publish src/ProcureAI.Api/ProcureAI.Api.csproj \
    -c Release --no-restore -o /out

FROM mcr.microsoft.com/dotnet/aspnet:10.0
WORKDIR /app
COPY --from=build /out/ .
ENV ASPNETCORE_HTTP_PORTS=8080
USER $APP_UID
EXPOSE 8080
ENTRYPOINT ["dotnet", "ProcureAI.Api.dll"]
```

deploy/Ai.Dockerfile

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY src/ProcureAI.Ai/ src/ProcureAI.Ai/
RUN dotnet restore src/ProcureAI.Ai/ProcureAI.Ai.csproj
RUN dotnet publish src/ProcureAI.Ai/ProcureAI.Ai.csproj \
    -c Release --no-restore -o /out

FROM mcr.microsoft.com/dotnet/aspnet:10.0
WORKDIR /app
COPY --from=build /out/ .
ENV ASPNETCORE_HTTP_PORTS=8080
USER $APP_UID
EXPOSE 8080
ENTRYPOINT ["dotnet", "ProcureAI.Ai.dll"]
```

Dacă adaugi proiecte referențiate sau Directory.Packages.props, adaptează COPY astfel încât restore să le vadă. Validează global.json, lock files și versiunile NuGet. În etapa de deploy folosește restore --locked-mode după generarea lock files.

deploy/Web.Dockerfile

```
FROM node:22-alpine AS build
WORKDIR /web
COPY src/procureai-web/package*.json ./
RUN npm ci
COPY src/procureai-web/ ./
RUN npm run build

FROM nginx:stable-alpine
COPY deploy/nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /web/dist/ /usr/share/nginx/html/
EXPOSE 80
```

MinIO: situația actuală a distribuției

Repository-ul MinIO Community este arhivat din 25 aprilie 2026; README-ul indică distribuție source-only. Ultimul release listat include un security fix din octombrie 2025. Pentru laborator reproducem acel tag din surse; pentru o instalare menținută alege un produs S3 suportat sau distribuția comercială după verificarea condițiilor. Nu presupune că minio/minio:latest este o versiune actualizată. [S26]

deploy/Minio.Dockerfile - build de laborator din tag explicit

```
FROM golang:1.25-bookworm AS build
ARG MINIO_VERSION=RELEASE.2025-10-15T17-29-55Z
ENV CGO_ENABLED=0 GOBIN=/out
RUN go install github.com/minio/minio@${MINIO_VERSION}

FROM debian:bookworm-slim
RUN apt-get update && apt-get install -y --no-install-recommends \
    ca-certificates curl && rm -rf /var/lib/apt/lists/*
COPY --from=build /out/minio /usr/local/bin/minio
RUN mkdir /data && chown 10001:10001 /data
USER 10001:10001
EXPOSE 9000 9001
ENTRYPOINT ["minio"]
CMD ["server", "/data", "--console-address", ":9001"]
```

Construirea unui tag arhivat nu oferă patch-uri viitoare. IObjectStore/S3 limitează impactul unei înlocuiri. Verifică licența și suportul pentru distribuția pe care o folosești; documentul nu tratează MinIO Community ca recomandare implicită pentru producție.

deploy/nginx.conf

```
server {
  listen 80;
  server_name _;
  root /usr/share/nginx/html;
  client_max_body_size 55m;

  location /api/ {
    proxy_pass http://api:8080;
    proxy_http_version 1.1;
    proxy_read_timeout 120s;
    proxy_buffering off;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-Proto $scheme;
  }

  location / {
    try_files $uri $uri/ /index.html;
  }
}
```

API-ul setează limita multipart la 50 MiB și limita Kestrel cu overhead suficient. Dacă implementezi range requests pentru audio/video, transmite corect Range, Content-Range și Accept-Ranges; Nginx nu poate crea suport Range într-un endpoint backend care nu îl implementează.

.dockerignore - la rădăcina build context

```
**/bin
**/obj
**/node_modules
**/dist
.git
.env
output
tmp
```

25 / Docker Compose: cinci containere + migrare one-shot

Fișierul de mai jos este configurația de referință a nucleului. migrate este o execuție temporară a imaginii API, nu o aplicație permanentă. MinIO și PostgreSQL au volume; AI nu are stare locală durabilă. depends_on cu service_healthy/service_completed_successfully ordonează bootstrap-ul, dar serviciile trebuie să gestioneze și indisponibilitatea ulterioară. [S31]

compose.yaml - definiții comune și stocare

```
name: procureai

x-api-env: &api-env
  ASPNETCORE_HTTP_PORTS: "8080"
  ConnectionStrings__App: >-
    Host=db;Database=procureai;Username=procureai;Password=${DB_PASSWORD:?set DB_PASSWORD}
  Storage__Endpoint: http://minio:9000
  Storage__AccessKey: ${MINIO_ROOT_USER:?set MINIO_ROOT_USER}
  Storage__SecretKey: ${MINIO_ROOT_PASSWORD:?set MINIO_ROOT_PASSWORD}
  Storage__Bucket: procureai
  Ai__BaseUrl: http://ai:8080/
  Internal__Key: ${INTERNAL_API_KEY:?set INTERNAL_API_KEY}
  Rag__Provider: ${RAG_PROVIDER:-AzureSearch}

services:
  db:
    image: postgres:17
    environment:
      POSTGRES_DB: procureai
      POSTGRES_USER: procureai
      POSTGRES_PASSWORD: ${DB_PASSWORD:?set DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U procureai -d procureai"]
      interval: 5s
      timeout: 3s
      retries: 15
    restart: unless-stopped

  minio:
    build:
      context: .
      dockerfile: deploy/Minio.Dockerfile
    environment:
      MINIO_ROOT_USER: ${MINIO_ROOT_USER:?set MINIO_ROOT_USER}
      MINIO_ROOT_PASSWORD: ${MINIO_ROOT_PASSWORD:?set MINIO_ROOT_PASSWORD}
    volumes:
      - objectdata:/data
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:9000/minio/health/live"]
      interval: 5s
      timeout: 3s
      retries: 15
    restart: unless-stopped
```

compose.yaml - continuarea aceluiași services

```
  ai:
    build:
      context: .
      dockerfile: deploy/Ai.Dockerfile
    environment:
      ASPNETCORE_HTTP_PORTS: "8080"
      Internal__Key: ${INTERNAL_API_KEY:?set INTERNAL_API_KEY}
      Storage__Endpoint: http://minio:9000
      Storage__AccessKey: ${MINIO_ROOT_USER:?set MINIO_ROOT_USER}
      Storage__SecretKey: ${MINIO_ROOT_PASSWORD:?set MINIO_ROOT_PASSWORD}
      Storage__Bucket: procureai
      Foundry__OpenAiEndpoint: ${FOUNDRY_OPENAI_ENDPOINT:?set endpoint}
      Foundry__ApiKey: ${FOUNDRY_API_KEY:?set key}
      Foundry__ProjectEndpoint: ${FOUNDRY_PROJECT_ENDPOINT:-}
      Foundry__ChatDeployment: ${CHAT_DEPLOYMENT:-procureai-chat}
      Foundry__EmbeddingDeployment: ${EMBEDDING_DEPLOYMENT:-procureai-embed}
      ContentUnderstanding__Endpoint: ${CU_ENDPOINT:?set CU_ENDPOINT}
      ContentUnderstanding__ApiKey: ${CU_API_KEY:?set CU_API_KEY}
      Search__Endpoint: ${SEARCH_ENDPOINT:?set SEARCH_ENDPOINT}
      Search__ApiKey: ${SEARCH_API_KEY:?set SEARCH_API_KEY}
      Search__Index: procureai-chunks
      Rag__Provider: ${RAG_PROVIDER:-AzureSearch}
      Backend__BaseUrl: http://api:8080/
    depends_on:
      minio:
        condition: service_healthy
    restart: unless-stopped

  migrate:
    build:
      context: .
      dockerfile: deploy/Api.Dockerfile
    environment: *api-env
    command: ["--migrate"]
    depends_on:
      db:
        condition: service_healthy
    restart: "no"

  api:
    build:
      context: .
      dockerfile: deploy/Api.Dockerfile
    environment: *api-env
    depends_on:
      migrate:
        condition: service_completed_successfully
      minio:
        condition: service_healthy
      ai:
        condition: service_started
    restart: unless-stopped

  web:
    build:
      context: .
      dockerfile: deploy/Web.Dockerfile
    ports:
      - "127.0.0.1:8080:80"
    depends_on:
      api:
        condition: service_started
    restart: unless-stopped

volumes:
  pgdata:
  objectdata:
```

Concatenează blocurile în același compose.yaml, fără un al doilea services. API, AI, DB și S3 nu au porturi publicate. Pentru readiness vezi capitolul 27; pentru pgvector vezi capitolul 17.

26 / Configurare, secrete și feature flags

.env.example - valorile sunt placeholders, nu credențiale reale

```
DB_PASSWORD=replace-with-a-long-local-password
MINIO_ROOT_USER=procureai-storage
MINIO_ROOT_PASSWORD=replace-with-a-long-storage-password
INTERNAL_API_KEY=replace-with-a-random-internal-secret

FOUNDRY_OPENAI_ENDPOINT=https://RESOURCE.openai.azure.com/openai/v1/
FOUNDRY_PROJECT_ENDPOINT=https://RESOURCE.services.ai.azure.com/api/projects/PROJECT
FOUNDRY_API_KEY=replace-with-resource-key
CHAT_DEPLOYMENT=procureai-chat
EMBEDDING_DEPLOYMENT=procureai-embed

CU_ENDPOINT=https://RESOURCE.cognitiveservices.azure.com
CU_API_KEY=replace-with-content-understanding-key
SEARCH_ENDPOINT=https://SEARCH.search.windows.net
SEARCH_API_KEY=replace-with-search-key
RAG_PROVIDER=AzureSearch
```

Copiezi în .env și înlocuiești fiecare placeholder. Cheile sunt pentru laborator și sunt accesibile numai pe server; nu comite .env. Exemplul folosește aceeași credențială root S3 pentru simplitatea bootstrap-ului; înainte de o instalare menținută separă privilegiile tehnice backend/gateway. Nu creezi utilizatori de aplicație.

Feature flags și limite în appsettings.json al gateway-ului

```
{
  "Capabilities": {
    "DocumentExtraction": true,
    "MediaUnderstanding": true,
    "AgentMode": true,
    "Mcp": false,
    "OpenApiTools": false,
    "FileSearch": false,
    "CodeInterpreter": false,
    "WebSearch": false,
    "SpeechOutput": false,
    "Translation": false,
    "LanguageAnalysis": false,
    "ImageGeneration": false,
    "VideoGeneration": false,
    "MultiAgent": true
  },
  "Limits": {
    "MaxToolRounds": 5,
    "MaxToolCalls": 5,
    "MaxRetrievedSources": 8,
    "MaxOutputTokens": 1200,
    "IngestionConcurrency": 1
  }
}
```

Flags sunt configurații ale aplicației, nu proprietăți native ale serviciului Microsoft. Ele documentează traseul complet de activare din capitolul 31. La true, validează la start deployment-uri, endpoint-uri și permisiuni necesare; o capabilitate indisponibilă produce 409/503 explicabil, nu fallback tăcut.

Entra / managed identity

În Azure folosește identitatea serviciului și rolurile minime pentru Foundry, modele și Search. Un container Docker local nu primește managed identity doar pentru că există pe calculator un az login. Pentru SDK-ul de proiect în container configurezi o credențială de dezvoltare compatibilă, de exemplu un service principal cu AZURE_TENANT_ID, AZURE_CLIENT_ID și AZURE_CLIENT_SECRET; în Azure treci la managed identity. Audience/scope diferă după serviciu. [S06, S13]

27 / Bootstrap și pașii de dezvoltare

A. Resursele Microsoft

• Creează resource și project Microsoft Foundry într-o regiune care suportă capabilitățile alese.

• Deploy procureai-chat și procureai-embed; notează numele exacte, endpoint-urile și cotele.

• Configurează Content Understanding și model mappings/analyzere conform cerințelor serviciului. Nu presupune că două deployment-uri sunt suficiente pentru orice analyzer.

• Creează Azure AI Search și indexul procureai-chunks; activează semantic ranking dacă îl folosești.

• Configurează content filters, identități/roluri și conectează Application Insights la proiect.

• Pentru extensii, adaugă Speech/Translator/Language și deployment-uri image/video numai când activezi adaptorul.

B. Implementarea locală

• Generează proiectele, fixează pachetele și creează entitățile/migrațiile.

• Implementează IObjectStore, upload-ul cu compensare și bootstrap-ul bucket-ului.

• Implementează job store/lease, gateway extraction, normalizare și indexare.

• Implementează retrieval separat de generation și validatorul de citări.

• Adaugă UI-ul pentru fișiere/chat/surse și endpoint-urile health.

• Completează Dockerfile-urile, Compose și .env; verifică end-to-end pe un document mic.

Pornire - după completarea implementării

```
cp .env.example .env
docker compose config --quiet
docker compose build
docker compose up -d
docker compose ps
docker compose logs --tail=100 migrate api ai
```

În Windows poți copia .env.example în Explorer/PowerShell. Browser: http://localhost:8080. docker compose config complet poate afișa secretele interpolate; folosește --quiet pentru validare și nu distribui output-ul cu credențiale.

service_started nu garantează readiness. Adaugă /health/live și /health/ready, bootstrap de bucket și retry limitat. Health check-urile nu trebuie să apeleze modele facturabile.

C. Proba funcțională

• Încarcă un PDF de 2-3 pagini cu un răspuns cunoscut; așteaptă Ready.

• Pune o întrebare acoperită de document și deschide citarea/pagina.

• Pune o întrebare fără dovezi; aplicația trebuie să explice lipsa informației.

• Repornește API/AI în timpul unui job; acesta trebuie să se reia.

• Șterge documentul și verifică că nu mai intră în răspunsuri.

28 / Securitate, observabilitate și controlul costurilor

Safety în fluxul de date

Tratează documentele ca date neîncrezătoare. Prompt Shields analizează prompturi și documente pentru atacuri directe/indirecte; content filters și Content Safety au alte roluri. Pentru text ascuns în imagini, evaluează calea multimodală și textul extras, nu presupune că un scan text-only acoperă automat toate imaginile. [S22]

• PII detection/redaction înainte de logging sau de folosirea datelor unde politica o cere.

• Allowlist pentru tipuri de fișiere, tool-uri și host-uri cloud; fără fetch de URL arbitrar.

• Raw text și fișiere nu se loghează implicit; redactează cheile, SAS-uri și datele sensibile.

• Originalele sunt private; browserul primește fluxul prin API și content disposition sigur.

• Fără aplicație de login: acces prin localhost/rețea privată, fără expunere anonimă publică.

Trace-uri utile

Un CorrelationId/trace context traversează web → backend → AI → provider. Înregistrează duratele upload/extraction/embedding/retrieval/generation, deployment-ul, prompt version, input/output tokens, tool name + status și job retry. Folosește OpenTelemetry și Application Insights; tracing-ul de proiect Foundry se conectează la resursa App Insights. [S18]

Comunicație internă și HTTP

Backend-ul trimite X-Internal-Key; middleware-ul AI folosește comparație constant-time și exclude cheia din loguri. /health/live poate fi exceptat. Nu publica portul AI. Pentru clientul Foundry setează timeout finit și propagă CancellationToken; retry-ul se adaugă explicit, evitând dublarea lui în worker și HTTP.

Client HTTP din DI

```
builder.Services.AddHttpClient<FoundryV1Client>(client =>
{
    client.Timeout = TimeSpan.FromSeconds(60);
});
```

Costuri: formulă, nu o sumă inventată

Estimare per răspuns / per corpus

```
Cost_raspuns = tokeni_input * tarif_input
            + tokeni_output * tarif_output
            + retrieval + tool_usage + speech/media_optional

Cost_ingestion = extracție_documente/audio/video
              + tokeni_embeddings * tarif_embeddings
              + reindexări

Cost_lunar = cost_fix_Search + compute_local/cloud
           + stocare + telemetry + total_inferență + total_ingestion
```

Aplică unitatea tarifului real, de exemplu preț per milion de tokeni sau per pagină/minut. Cotele și un budget alert nu sunt automat un hard spending cap. Introdu în aplicație limite de fișier, concurență, output, tool rounds și rate limits; pentru media costisitoare limitează explicit joburile. Prețurile exacte se verifică înainte de deploy.

29 / Evaluări, teste și CI/CD pentru domeniu

Scenariu	Rezultat cerut
Oferte în monede diferite	NeedsInput; nu convertește folosind un curs inventat.
Agent de comparație schimbă scorul	Validatorul respinge; scorul backend rămâne autoritatea.
Un agent eșuează după checkpoint	Retry reutilizează pașii validați, fără reexecuție completă.
Lipsește transportul	Oferta este incompletă; nu presupune gratuit.
Supplier PDF conține instrucțiuni	Nu poate schimba tool allowlist sau criteriile.
Buget depășit	Run eșuează controlat; usage și pașii rămân disponibili.

Dataset și măsurare

Pregătește 20-30 cazuri seed, cu expected fields/actions și evidence IDs etichetate. Măsoară groundedness, corectitudinea citărilor, exhaustivitatea pe câmpuri/obligații, rata erorilor blocante și efectele tools. Pentru fiecare deployment/promptVersion păstrează un baseline reproductibil; obiectivul este compararea, nu o medie universală.

Foundry evaluators completează validările deterministe; nu înlocuiesc aserțiile pentru sume, scope, idempotency sau versiuni. Verifică scoruri de safety și prompt injection pe date seed. Trace-urile conțin corelație/usage, iar prompturile și documentele integrale sunt excluse implicit. [S17-S18, S22]

Verificări CI de pornire

```
dotnet restore
dotnet build --no-restore
dotnet test --no-build
# în src/procureai-web:
npm ci
npm run build
# la root, cu .env de laborator:
docker compose config --quiet
docker compose build
```

Validarea livrabilului PDF verifică lizibilitatea, JSON-urile și structura YAML. Mediul de redactare nu are runtime .NET/Docker pentru compilare/rulare. Smoke test-ul Azure și testele de integrare de aici se execută după implementarea completă.

30 / Diagnostic: probleme probabile și remediere

Simptom	Verificare / remediere
Foundry 401/403	Cheie/audience/roluri și endpoint; o identitate locală nu este automat disponibilă în container.
Foundry 404	model = numele deployment-ului; baza /openai/v1/ corectă; nu amesteca project/classic endpoints.
CU acceptă, dar nu are rezultat	202 înseamnă job pornit; citește Operation-Location și poll până la rezultat/error.
Azure nu citește fișierul MinIO	URL-ul intern nu este accesibil cloud. Trimite bytes sau folosește transfer temporar compatibil.
Search întoarce puține/zero rezultate	Index/chunk-uri, dimensiuni/model embeddings, filtre, limbă, starea documentului și generația.
Surse vechi după ștergere	Backend-ul validează Ready + generație înainte de prompt; cleanup index este eventual.
DbContext: second operation	Nu partaja contextul între operații concurente; factory/scope separat și await complet.
Job blocat după restart	Lease expirat și claim/retry; ProviderOperation trebuie să fie persistat.
Upload 413	Limitele Nginx, Kestrel și FormOptions trebuie aliniate și să permită overhead multipart.
SPA apare în locul fișierului	Ruta /api/content merge la backend; obiectele S3 nu sunt fișiere statice în dist.
Vite funcționează numai pe host	Dev server 0.0.0.0, port/proxy corect; build final servit prin Nginx.
MinIO imagine/tag indisponibil	Build din surse tag explicit; nu presupune distribuție nouă minio/minio:latest.
Quota / cost excesiv	Limite de concurență/context/tools, Retry-After, job ID pentru media și evaluări separate de trafic.

Oprire și backup

docker compose down oprește containerele și păstrează volumele. down -v șterge volumele și este o operație de reset intenționată. Backup-ul acoperă PostgreSQL și obiectele S3; testează restaurarea și păstrează configurația indexului pentru reindexare. Nu ai nevoie de o funcție de backup în UI.

31 / Ordinea implementării și acoperirea AI-103

Etapă	Rezultat verificabil
1 / Infrastructură	5 containere, migrare one-shot, health, EF și S3.
2 / Documente	Upload, hash, joburi durabile și extragere cu proveniență.
3 / Domeniu	Schema/API din cap. 5-8 și regulile .NET din cap. 33.
4 / Foundry	Prompt/DTO/validator, tools, grounding și buget.
5 / UI	Panou unic, polling, rezultate/citări și acțiunea specifică.
6 / Calitate	Dataset, teste deterministe, safety și tracing corelat.
7 / Laborator	RAG alternativ, MCP/OpenAPI, multi-agent și media adapters.

Arie oficială	Exercițiul din acest proiect
Planificare / administrare	Cap. 2-4, 24-30: deployment, identity, CI, health, cost și observabilitate.
Generativ / agenți	Cap. 12-21, 33-35: RAG, Responses, tools, prompt, memorie și orchestration.
Computer vision	Cap. 11, 18, 22: imagini, evidență vizuală, generare/editare și video activabil.
Text / speech	Cap. 11, 15, 22, 28: extragere, PII, sentiment, rezumat, traducere și STT/TTS.
Extragerea informației	Cap. 9-14, 18, 32: layout, câmpuri, index, skills și proveniență.

Harta folosește obiectivele oficiale AI-103, nu chei de răspuns ale unui simulator. Laboratoarele .NET exersează conceptele; programa menționează și dezvoltare Python, deci acest PDF .NET nu înlocuiește exercițiile Python pentru examen. Nu se promite utilizarea tuturor conectorilor sau deployment-urilor din catalog. [S01]

Skills și multimodal ca laborator

Pipeline-ul principal încarcă bytes din MinIO în extractor și face push indexing. Un Azure AI Search indexer nu citește automat S3 local. Pentru Text Split/OCR/embedding/custom skills folosește staging Azure Blob sau un adaptor explicit. Speech custom training, fine-tuning și image/video generation sunt extensii separate activabile, nu condiții de startup. [S32]

32 / Fluxul end-to-end și limita fiecărui pas

Pas	Acțiune
1	NecesarAgent produce cerințe structurate validate de backend.
2	FurnizoriAgent adună exclusiv evidența permisă pentru fiecare ofertă.
3	ComparațieAgent verifică acoperirea; normalizarea numerică este în cod.
4	score_offers calculează cost/termen/conformitate cu ponderi fixe.
5	RecomandareAgent redactează concluzia din scoruri și surse existente.
6	Validatorul verifică citările, câștigătorul și necunoscutele; salvează raportul.

Stările resursei

Created / Extracting / Planning / Evaluating / Recommending / Completed / NeedsInput / Failed

Tranziții și checkpoint

API-ul salvează o resursă și job-ul în aceeași tranzacție scurtă. Worker-ul revendică job-ul, construiește snapshot-ul și apelează gateway-ul fără tranzacție DB deschisă. După fiecare pas validat persistă rezultatul, usage și referințele. Un lease expirat permite reluarea; cleanup/retry utilizează OperationId stabil.

Ready pentru document, rezultat pentru domeniu

Un document Ready indică extracție/indexare disponibile. O resursă de domeniu Completed/Validated/Answered este o stare separată. Dacă originalul este șters ori snapshot-ul se schimbă, rezultatul se marchează stale și nu se prezintă ca analiză actuală.

Patru roluri distincte sunt vizibile în progres; scorurile se reproduc din datele seed, recomandarea citează ofertele și un restart reia run-ul fără achiziții externe.

Orchestrator durabil în monolit

Pseudocod C# de serviciu: o etapă validată la fiecare checkpoint

```
string[] roles = ["needs", "suppliers", "compare", "recommend"];
foreach (string role in roles)
{
    var input = await snapshots.ForStepAsync(runId, role, ct);
    var cached = await steps.FindSucceededAsync(runId, role,
        input.Hash, promptVersion, ct);
    if (cached is not null) continue;
    await budgets.EnsureAvailableAsync(runId, ct);
    var result = await ai.ExecuteStepAsync(new DomainStepRequest(
        correlationId, operationId, "ProcureAI", runId, role,
        input.Hash, promptVersion, input.Json, input.DocumentIds), ct);
    var validated = validators.Validate(role, result, input);
    await steps.SaveSucceededAsync(runId, role, input.Hash,
        promptVersion, validated, result.InputTokens,
        result.OutputTokens, ct);
}
```

snapshots, steps, budgets și validators sunt servicii de implementat în API. În AI, fiecare role mapează la instrucțiuni/deployment/tools separate. Fournizorii folosesc snapshot-ul rezultatului needs; compare folosește ofertele validate; recommend primește scorul .NET. Un agent care întoarce needs_input oprește etapele dependente. Persistarea rezultatului și a usage-ului se face atomic cu starea AgentStep.

Variantă Microsoft Agent Framework pentru laborator

C# - pipeline în același AI gateway

```
using Microsoft.Agents.AI;
using Microsoft.Agents.AI.Workflows;
using Microsoft.Extensions.AI;

// foundryChatClient: IChatClient configurat cu Microsoft Foundry.
var needs = new ChatClientAgent(foundryChatClient,
    "Extrage cerințele în JSON, fără valori inventate.", "NeedsAgent");
var suppliers = new ChatClientAgent(foundryChatClient,
    "Folosește doar ofertele și dovezile permise.", "SuppliersAgent");
var compare = new ChatClientAgent(foundryChatClient,
    "Compară ofertele validate; nu recalcula scoruri.", "CompareAgent");
var recommend = new ChatClientAgent(foundryChatClient,
    "Explică scorul .NET cu citări existente.", "RecommendAgent");
var workflow = AgentWorkflowBuilder.BuildSequential(
    [needs, suppliers, compare, recommend]);
await using var execution = await InProcessExecution.RunStreamingAsync(
    workflow, new List<ChatMessage> { new(ChatRole.User, inputJson) });
await execution.TrySendMessageAsync(new TurnToken(emitEvents: true));
await foreach (WorkflowEvent evt in execution.WatchStreamAsync())
{
    if (evt is WorkflowOutputEvent output)
    {
        var messages = output.As<List<ChatMessage>>();
        // Validează JSON/evidence și returnează DTO-ul către API.
    }
}
```

Fragmentul arată forma SDK-ului documentată pentru C#. Completează tool bindings, foundryChatClient, policy și checkpoint adapter. Execuția InProcess nu este durabilă doar pentru că utilizează Workflows. Configurează identitatea pentru endpoint-ul proiectului și fixează pachetele Microsoft.Agents.AI / Workflows și extensiile compatibile din exemplul oficial. BuildSequential transmite implicit contextul anterior; minimizează contextul prin DTO-uri și executori. [S40-S42]

33 / Logica .NET specifică și reguli deterministe

Fragment de implementare al regulii centrale

```
public sealed record OfferValue(Guid Id, string Currency,
    decimal Net, decimal Shipping, int LeadDays, bool MeetsRequirements);
public sealed record OfferScore(Guid Id, decimal Cost, decimal Score);

public static IReadOnlyList<OfferScore> Score(
    IReadOnlyList<OfferValue> offers, string currency, int maxDays)
{
    if (offers.Count == 0 || offers.Any(x => x.Currency != currency ||
        x.Net < 0 || x.Shipping < 0 || x.LeadDays < 0) || maxDays <= 0)
        throw new ArgumentException("incomparable_offers");
    var eligible = offers.Where(x => x.MeetsRequirements &&
        x.LeadDays <= maxDays).ToArray();
    if (eligible.Length == 0) return [];
    decimal minCost = eligible.Min(x => x.Net + x.Shipping);
    if (minCost <= 0) throw new ArgumentException("cost_required");
    return eligible.Select(x => new OfferScore(x.Id, x.Net + x.Shipping,
        0.7m * (minCost / (x.Net + x.Shipping)) +
        0.3m * (1m - ((decimal)x.LeadDays / maxDays))))
        .OrderByDescending(x => x.Score).ThenBy(x => x.Cost)
        .ThenBy(x => x.Id).ToArray();
}
```

Roluri separate în același container

NecesarAgent, FurnizoriAgent, ComparațieAgent și RecomandareAgent au prompturi și tool allowlists separate. Separarea rolurilor nu cere câte un proces sau container. Toate inferențele folosesc deployments Microsoft Foundry.

Orchestrare și checkpoint

API-ul revendică un job și apelează etapa următoare prin /internal/agents/step. Gateway-ul nu citește DB. API-ul validează și persistă OutputJson, usage și InputHash pentru fiecare AgentStep. Dacă procesul cade, etapa validată se reutilizează; un apel model repetat poate costa din nou.

Paralelism controlat

Poți extrage/evalua oferte independente cu maximum două apeluri concomitente. Fiecare persistență folosește context EF propriu. Planificarea și recomandarea rămân secvențiale. Task.WhenAll de apeluri LLM nu înlocuiește checkpoint-urile și nu oferă reluare după restart.

Grila exemplului

Se folosește preț net + transport, aceeași monedă și cantitate/bază fiscală, eligibilitate booleană verificată, termen maxim obligatoriu și ponderi 0.7 cost / 0.3 termen. Datele incomplete blochează calculul. Moneda și ponderile se fixează în snapshot; agenții nu le negociază.

Contractul serviciilor și DI

Fragmentele reprezintă metode de servicii sau funcții helper. Adaugă namespace-urile, AppDbContext factory, DTO validators și maparea ProblemDetails. Configurează typed HttpClient pentru gateway; CancellationToken curge de la worker/request la HTTP/S3/EF. Nu construiește HttpClient la fiecare cerere.

Înregistrări de referință - completează clasele serviciilor

```
builder.Services.AddDbContextFactory<AppDbContext>(options =>
    options.UseNpgsql(builder.Configuration.GetConnectionString("App")));
builder.Services.AddHttpClient<DomainAiClient>(client => {
    client.BaseAddress = new Uri("http://ai:8080/");
    client.Timeout = TimeSpan.FromSeconds(120);
});
builder.Services.AddScoped<ProcureAIService>();
builder.Services.AddHostedService<DurableDomainWorker>();
#if DEBUG
// Fixtures artificiale: se citesc o dată, nu se adaugă un serviciu extern.
#endif
```

DomainAiClient, serviciul de domeniu și worker-ul sunt clase de implementat. Worker singleton nu injectează un DbContext scoped; folosește factory ori un scope pe iterație. Nu retry automat o scriere externă ambiguă. Pentru pașii model folosește retry limitat la erori tranzitorii și înregistrează posibilul cost al apelului repetat.

Configurația domeniului în API

appsettings.Domain.json - configurație a aplicației

```
{
  "Domain": "ProcureAI",
  "PipelineVersion": "domain-v1",
  "PromptVersion": "prompt-v1",
  "MaxToolRounds": 5,
  "MaxParallelAiCalls": 2,
  "MaxRunTokens": 15000,
  "ExternalWrites": false
}
```

Aceste chei sunt inventarul de configurare al aplicației, nu proprietăți native Foundry. Leagă-le prin IOptions și validează la startup. MaxRunTokens include toate etapele/tool rounds; usage-ul real se acumulează în DB.

34 / Scenarii de test și acceptare end-to-end

Cazul de bază

Necesar seed: 10 unități, monedă EUR, livrare ≤10 zile. Oferta A are net 1000 + transport 50, 7 zile; B are net 980 + transport 120, 9 zile. costul comparabil este 1050 vs 1100; .NET stabilește scorurile și modelul explică alegerea.

Verificare	Dovadă
Persistență	Restart api/ai nu pierde document, rezultat sau job.
Proveniență	Fiecare câmp/afirmație verificabilă are source IDs permise și link valid.
Izolare	AI nu se conectează la PostgreSQL; browserul nu primește chei.
Retry	Job reluat nu dublează rezultat/snapshot/acțiune locală.
Erori	Provider 429/503 produce backoff limitat; UI afișează code/traceId sigur.
Cost	usage este acumulat pe pași și deadline/budget opresc bucla.

Teste de integrare necesare

• PostgreSQL real pentru migrații, unique constraints, concurrency și lease/reclaim. Nu valida SQL cu EF InMemory.

• S3 real în Compose pentru upload/hash/content și cleanup după eșec.

• AI gateway cu fake HTTP deterministic pentru parsare, incomplete response și tools limit.

• Un smoke test separat cu resurse Foundry reale pentru endpoint/auth/model/extractor și un fișier seed.

CI implicit nu cheamă Azure pentru fiecare build. Smoke test-ul live se execută cu chei/identity autorizate și buget; documentele sunt artificiale. Păstrează baseline-ul în repository, versiunea promptului și un raport simplu JSON. Nu adăuga un dashboard de evaluare în produs.

35 / Frontend și pașii concreți de dezvoltare

Zonă UI	Comportament
Intrare	Upload comun + câmpurile de domeniu definite în cap. 8.
Procesare	Buton explicit; 202 -> polling; busy/error afișate clar.
Rezultat	Date structurate, stări, necunoscute și explicația AI.
Surse	Original stream prin API; pagină/timestamp și text citat.
Acțiune finală	Confirmare locală / export JSON / raport / comparație, după temă.

src/domain-api.ts - helper TypeScript minimal

```
export async function postJson<T>(path: string, body: unknown,
  signal?: AbortSignal): Promise<T> {
  const r = await fetch(path, { method: "POST", signal,
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(body) });
  if (!r.ok) throw new Error(`Request failed: ${r.status}`);
  return r.json();
}
export async function loadDomain<T>(id: string,
  signal?: AbortSignal): Promise<T> {
  const r = await fetch(`/api/procurement/runs/${id}`, { signal });
  if (!r.ok) throw new Error(`Load failed: ${r.status}`);
  return r.json();
}
```

Implementare în ordine

• Creează soluția .NET 10, două web projects Api/Ai și React Vite TS; fixează dependencies și lockfiles.

• Implementează entitățile/ToTable/constraints și InitialSchema. Migrate one-shot trebuie să ruleze înainte de API.

• Construiește S3 storage, upload, jobs, worker și extractorul principal. Adu seed-urile artificiale în repository.

• Implementează API-ul domeniului și regulile deterministe fără model. Verifică scenariul din cap. 34.

• Leagă Foundry: model, prompt/DTO, tools, scope/evidence, usage și safety. Adaugă un test live controlat.

• Completează panoul React și polling; apoi rulează Docker Compose cu volume și restart test.

• Activează separat laboratorul MCP/OpenAPI/RAG/media și compară evaluările cu baseline-ul.

Definition of done: Patru roluri distincte sunt vizibile în progres; scorurile se reproduc din datele seed, recomandarea citează ofertele și un restart reia run-ul fără achiziții externe. Nu există conturi de utilizator și nu se introduc containere permanente suplimentare.

36 / Surse oficiale și limitele livrabilului

Documentație oficială consultată la 11 octombrie 2026. Alegerile de arhitectură, limitele, grilele, numele DTO-urilor și exemplele sunt propuneri pentru proiect. Disponibilitatea SDK-urilor, a preview-urilor și a deployment-urilor se verifică în regiunea/tenant-ul ales.

[S01] Programa oficială AI-103
Deschide documentația oficială

[S02] .NET 10 - noutăți și LTS
Deschide documentația oficială

[S03] EF Core 10 - cerințe și noutăți
Deschide documentația oficială

[S04] Provider Npgsql pentru EF Core
Deschide documentația oficială

[S05] Vite - Getting Started
Deschide documentația oficială

[S06] Foundry SDK nou - C# și autentificare
Deschide documentația oficială

[S07] Foundry project Responses API
Deschide documentația oficială

[S08] Azure OpenAI Responses API în Foundry
Deschide documentația oficială

[S09] Azure AI Search - vector search
Deschide documentația oficială

[S10] Content Understanding - analyzeBinary REST GA
Deschide documentația oficială

[S11] Content Understanding - analyzere prebuilt
Deschide documentația oficială

[S12] Agentic retrieval - GA și preview
Deschide documentația oficială

[S13] Embeddings în Foundry - .NET/REST
Deschide documentația oficială

[S14] Function calling pentru agenți Foundry
Deschide documentația oficială

[S15] Foundry Toolbox
Deschide documentația oficială

[S16] Conectarea agenților la MCP
Deschide documentația oficială

[S17] Evaluări în Microsoft Foundry
Deschide documentația oficială

[S18] Tracing pentru agenți Foundry
Deschide documentația oficială

[S19] Azure Speech în Foundry Tools
Deschide documentația oficială

[S20] Azure Translator în Foundry Tools
Deschide documentația oficială

[S21] Sora 2 - video generation / remix (preview)
Deschide documentația oficială

[S22] Prompt Shields
Deschide documentația oficială

[S23] DbContext lifetime și thread safety
Deschide documentația oficială

[S24] Aplicarea migrațiilor EF Core
Deschide documentația oficială

[S25] ASP.NET Core - BackgroundService/hosted services
Deschide documentația oficială

[S26] MinIO Community - repository și distribuție
Deschide documentația oficială

[S26a] MinIO - taguri și security release
Deschide documentația oficială

[S27] pgvector - extensia PostgreSQL
Deschide documentația oficială

[S28] pgvector-dotnet - integrarea .NET/EF
Deschide documentația oficială

[S29] AWS SDK pentru .NET v4 - client S3
Deschide documentația oficială

[S30] Content Understanding - GA / agentic preview
Deschide documentația oficială

[S31] Docker Compose - startup order și health
Deschide documentația oficială

[S32] Azure AI Search - skills pentru enrichment
Deschide documentația oficială

[S33] Azure Language în Foundry Tools
Deschide documentația oficială

[S34] Modele Azure în Microsoft Foundry
Deschide documentația oficială

[S35] GPT Image API - parametri și măști
Deschide documentația oficială

[S36] Document Intelligence - invoice v4 GA
Deschide documentația oficială

[S37] Document Intelligence - receipt v4 GA
Deschide documentația oficială

[S38] SDK .NET Document Intelligence
Deschide documentația oficială

[S39] Foundry Agent Service - opțiuni actuale
Deschide documentația oficială

[S40] Agent Framework - workflow secvențial C#
Deschide documentația oficială

[S41] Agent Framework - workflow concurent C#
Deschide documentația oficială

[S42] Modele de orchestrare a agenților
Deschide documentația oficială

[S43] Content Understanding - documente și grounding
Deschide documentația oficială

[S44] Document Intelligence REST - analiză v4
Deschide documentația oficială

Livrabil și validare

Este livrat PDF-ul de specificație, cu fragmente de cod și configurații. Nu este livrat un repository complet compilat și nu sunt provisionate resurse Azure. JSON și Compose sunt verificate structural, paginile sunt randate și inspectate. Build-ul, testele și verificarea efectivă Foundry se fac după completarea implementării.